

```
In [1]: import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import roc_auc_score
```

```
In [2]: df = pd.read_csv("train.csv")
df.head()
```

```
Out[2]:
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	I
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	I
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	I

```
In [3]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    object
4   Sex          891 non-null    object
5   Age          714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    object
9   Fare         891 non-null    float64
10  Cabin        204 non-null    object
11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

To start, I will need to determine which feature to use.

PassengerID, Name, Ticket can be dropped first since they have very small influence on surviving. Also dropped Cabin since it has many null values.

```
In [4]: df = df.drop(columns=["PassengerId", "Ticket", "Name", "Cabin"])
```

```
In [5]: # performing one-hot encoding
df = pd.get_dummies(df, columns=["Sex", "Embarked"], drop_first=True)
```

```
In [6]: #handle missing age
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Fare"] = df["Fare"].fillna(df["Fare"].median())
```

```
In [7]: df.head()
```

```
Out[7]:
```

	Survived	Pclass	Age	SibSp	Parch	Fare	Sex_male	Embarked_Q	Embarked_S
0	0	3	22.0	1	0	7.2500	1	0	1
1	1	1	38.0	1	0	71.2833	0	0	0
2	1	3	26.0	0	0	7.9250	0	0	1
3	1	1	35.0	1	0	53.1000	0	0	1
4	0	3	35.0	0	0	8.0500	1	0	1

```
In [8]: features = [
    "Pclass",
    "Age",
    "Fare",
    "Sex_male",
    "Embarked_Q",
    "Embarked_S"
]
```

```
X = df[features]
y = df["Survived"]
```

```
In [9]: model = LogisticRegression(max_iter=1000)
model.fit(X, y)

pd.Series(model.coef_[0], index=X.columns).sort_values()
```

```
Out[9]: Sex_male      -2.488470
Pclass      -1.131345
Embarked_S   -0.489260
Age          -0.032808
Embarked_Q   -0.002370
Fare         0.000039
dtype: float64
```

```
In [10]: model_2 = LogisticRegression(penalty="l1", solver="liblinear")
model_2.fit(X, y)

pd.Series(model_2.coef_[0], index=X.columns)
```

```
Out[10]: Pclass      -1.078796
Age          -0.030227
Fare         0.000558
Sex_male     -2.502369
Embarked_Q   0.000000
Embarked_S   -0.434332
dtype: float64
```

```
In [11]: #Dropped Embarked_Q due to l1
features = ["Sex_male", "Pclass", "Age", "Fare", "Embarked_S"]
X = df[features]
y = df["Survived"]
```

```
In [12]: X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
In [13]: pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(
        penalty="l1",
        solver="liblinear",
        C=1.0
    ))
])
```

```
In [14]: pipe.fit(X_train, y_train)
```

```
Out[14]: Pipeline(steps=[('scaler', StandardScaler()),
    ('model',
    LogisticRegression(penalty='l1', solver='liblinear'))])
```

```
In [15]: y_pred = pipe.predict(X_val)
accuracy_score(y_val, y_pred)
```

Out[15]: 0.770949720670391

```
In [16]: confusion_matrix(y_val, y_pred)
```

```
Out[16]: array([[92, 18],
               [23, 46]])
```

```
In [17]: y_prob = pipe.predict_proba(X_val)[:, 1]
         roc_auc_score(y_val, y_prob)
```

Out[17]: 0.8366271409749672

```
In [18]: coefs = pipe.named_steps["model"].coef_[0]
         pd.Series(coefs, index=features).sort_values()
```

```
Out[18]: Sex_male      -1.228315
         Pclass        -0.958150
         Age           -0.437615
         Embarked_S    -0.248938
         Fare           0.000000
         dtype: float64
```

```
In [19]: dtest_raw = pd.read_csv("test.csv")
```

```
In [20]: dtest = dtest_raw.drop(columns=["PassengerId", "Ticket", "Name", "Cabin"])

         dtest = pd.get_dummies(
             dtest,
             columns=["Sex", "Embarked"],
             drop_first=True
         )

         dtest["Age"] = dtest["Age"].fillna(dtest["Age"].median())
         dtest["Fare"] = dtest["Fare"].fillna(dtest["Fare"].median())
```

```
In [21]: features = ["Pclass", "Age", "Fare", "Sex_male", "Embarked_S"]
         X_test = dtest[features]
```

```
In [22]: test_pred = pipe.predict(X_test)
         test_prob = pipe.predict_proba(X_test)[:, 1]
```

```
In [23]: submission = pd.DataFrame({
         "PassengerId": dtest_raw["PassengerId"],
         "Survived": test_pred
     })
```

```
In [24]: submission.to_csv("submission.csv", index=False)
```

Got a score of 62.20 Now I will need to identify strong features

```
In [25]: df = pd.read_csv("train.csv")
         df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
         df["IsAlone"] = (df["FamilySize"] == 1).astype(int)
         df["Title"] = df["Name"].str.extract(r' ([A-Za-z])\.', expand=False)
```

```
df["Title"] = df["Title"].replace(
    ["Lady", "Countess", "Capt", "Col", "Don", "Dr", "Major", "Rev", "Sir", "Jonkheer",
     "Rare"]
)
df["Title"] = df["Title"].replace({"Mlle": "Miss", "Ms": "Miss", "Mme": "Mrs"})
```

```
In [26]: df["Age"] = df["Age"].fillna(df["Age"].median())
df["Fare"] = df["Fare"].fillna(df["Fare"].median())
df["Embarked"] = df["Embarked"].fillna("S")

# Drop columns no longer needed
df = df.drop(columns=["Name", "Ticket", "Cabin"])

# One-hot encode categorical features
df = pd.get_dummies(
    df,
    columns=["Sex", "Embarked", "Title"],
    drop_first=True
)
```

```
In [27]: X = df.drop(columns=["Survived", "PassengerId"])
y = df["Survived"]
```

Instead of logarithmic regression, i will use random forest.

```
In [28]: from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=500,
    max_depth=6,
    min_samples_split=10,
    random_state=42
)

rf.fit(X, y)
```

```
Out[28]: RandomForestClassifier(max_depth=6, min_samples_split=10, n_estimators=500,
                                random_state=42)
```

```
In [29]: importances = pd.Series(
    rf.feature_importances_,
    index=X.columns
).sort_values(ascending=False)

importances
```

```
Out[29]: Title_Mr      0.218701
Sex_male    0.205232
Fare        0.117765
Pclass      0.107691
Age         0.065931
Title_Miss  0.064197
Title_Mrs   0.062951
FamilySize  0.059025
SibSp       0.034096
Parch       0.019375
Embarked_S  0.014742
IsAlone     0.011829
Title_Rare  0.010032
Embarked_Q  0.008433
dtype: float64
```

```
In [30]: strong_features = importances[importances > 0.01].index.tolist()
X_strong = X[strong_features]
X_strong
```

```
Out[30]:
```

	Title_Mr	Sex_male	Fare	Pclass	Age	Title_Miss	Title_Mrs	FamilySize	SibSp	Parch
0	1	1	7.2500	3	22.0	0	0	2	1	0
1	0	0	71.2833	1	38.0	0	1	2	1	0
2	0	0	7.9250	3	26.0	1	0	1	0	0
3	0	0	53.1000	1	35.0	0	1	2	1	0
4	1	1	8.0500	3	35.0	0	0	1	0	0
...	...	...	...	...	...	...	...	...	...	...
886	0	1	13.0000	2	27.0	0	0	1	0	0
887	0	0	30.0000	1	19.0	1	0	1	0	0
888	0	0	23.4500	3	28.0	1	0	4	1	0
889	1	1	30.0000	1	26.0	0	0	1	0	0
890	1	1	7.7500	3	32.0	0	0	1	0	0

891 rows x 13 columns

```
In [31]: X_train, X_val, y_train, y_val = train_test_split(
X_strong, y, test_size=0.2, random_state=42, stratify=y
)
```

```
In [32]: rf_strong = RandomForestClassifier(
n_estimators=2000,
max_depth=7,
min_samples_split=10,
random_state=42
)

rf_strong.fit(X_train, y_train)
```

```
Out[32]: RandomForestClassifier(max_depth=7, min_samples_split=10, n_estimators=2000,
                                random_state=42)
```

```
In [33]: rf_pred = rf_strong.predict(X_val)
rf_acc = accuracy_score(y_val, rf_pred)

rf_acc
```

```
Out[33]: 0.8324022346368715
```

```
In [36]: from sklearn.ensemble import GradientBoostingClassifier

gb_strong = GradientBoostingClassifier(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=3,
    random_state=42
)

gb_strong.fit(X_train, y_train)
gb_strong.fit(X_train, y_train)
```

```
Out[36]: GradientBoostingClassifier(learning_rate=0.05, n_estimators=300,
                                      random_state=42)
```

```
In [37]: gb_pred = gb_strong.predict(X_val)
gb_acc = accuracy_score(y_val, gb_pred)

gb_acc
```

```
Out[37]: 0.8156424581005587
```

```
In [38]: dtest_raw = pd.read_csv("test.csv")
```

```
In [39]: dtest = dtest_raw.copy()
dtest["FamilySize"] = dtest["SibSp"] + dtest["Parch"] + 1
dtest["IsAlone"] = (dtest["FamilySize"] == 1).astype(int)
dtest["Title"] = dtest["Name"].str.extract(r' ([A-Za-z]+)\.', expand=False)

dtest["Title"] = dtest["Title"].replace(
    ["Lady", "Countess", "Capt", "Col", "Don", "Dr", "Major", "Rev", "Sir", "Jonkheer",
     "Rare"]
)
dtest["Title"] = dtest["Title"].replace({"Mlle": "Miss", "Ms": "Miss", "Mme": "Mrs"}
```

```
In [40]: dtest["Age"] = dtest["Age"].fillna(dtest["Age"].median())
dtest["Fare"] = dtest["Fare"].fillna(dtest["Fare"].median())
dtest["Embarked"] = dtest["Embarked"].fillna("S")

# Drop columns no longer needed
dtest = dtest.drop(columns=["Name", "Ticket", "Cabin"])

# One-hot encode categorical features
dtest = pd.get_dummies(
    dtest,
    columns=["Sex", "Embarked", "Title"],
```

```
    drop_first=True
)
dtest = dtest.drop(columns=["PassengerId"])
dtest = dtest[strong_features]
```

```
In [41]: test_pred = rf_strong.predict(dtest)
test_prob = rf_strong.predict_proba(dtest)[:, 1]
```

```
In [42]: submission = pd.DataFrame({
    "PassengerId": dtest_raw["PassengerId"],
    "Survived": test_pred
})
```

```
In [43]: submission.to_csv("submission_rf.csv", index=False)
```

Random forest give a accuracy score of 0.78229

```
In [44]: test_pred = gb_strong.predict(dtest)
test_prob = gb_strong.predict_proba(dtest)[:, 1]
submission = pd.DataFrame({
    "PassengerId": dtest_raw["PassengerId"],
    "Survived": test_pred
})
submission.to_csv("submission_gb.csv", index=False)
```

```
In [45]: rf = RandomForestClassifier(
    n_estimators=1000,
    max_depth=7,
    min_samples_split=5,
    min_samples_leaf=2,
    max_features="sqrt",
    random_state=42
)
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_val)
rf_acc = accuracy_score(y_val, rf_pred)

rf_acc
```

```
Out[45]: 0.8268156424581006
```

```
In [46]: gb = GradientBoostingClassifier(
    n_estimators=600,
    learning_rate=0.03,
    max_depth=3,
    subsample=0.8,
    random_state=42
)
gb.fit(X_train, y_train)
gb_pred = gb.predict(X_val)
gb_acc = accuracy_score(y_val, gb_pred)

gb_acc
```

```
Out[46]: 0.8324022346368715
```

```
In [47]: test_pred = gb.predict(dtest)
test_prob = gb.predict_proba(dtest)[: , 1]
submission = pd.DataFrame({
    "PassengerId": dtest_raw["PassengerId"],
    "Survived": test_pred
})
submission.to_csv("submission_gb_improved.csv", index=False)
```

```
In [48]: from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV

param_dist = {
    "n_estimators": [300, 500, 800, 1000],
    "max_depth": [None, 5, 6, 7, 8, 10],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4],
    "max_features": ["sqrt", "log2"]
}
```

```
In [52]: rf = RandomForestClassifier(random_state=42)

random_search = RandomizedSearchCV(
    estimator=rf,
    param_distributions=param_dist,
    n_iter=400,                # number of random configs tried
    scoring="accuracy",
    cv=5,                     # 5-fold CV
    n_jobs=-1,                # use all cores
    random_state=42
)

random_search.fit(X_train, y_train)
```

```
Out[52]: RandomizedSearchCV(cv=5, estimator=RandomForestClassifier(random_state=42),
                             n_iter=400, n_jobs=-1,
                             param_distributions={'max_depth': [None, 5, 6, 7, 8, 10],
                                                    'max_features': ['sqrt', 'log2'],
                                                    'min_samples_leaf': [1, 2, 4],
                                                    'min_samples_split': [2, 5, 10],
                                                    'n_estimators': [300, 500, 800, 100
0]},
                             random_state=42, scoring='accuracy')
```

```
In [53]: best_rf = random_search.best_estimator_
random_search.best_params_
```

```
Out[53]: {'n_estimators': 300,
          'min_samples_split': 10,
          'min_samples_leaf': 4,
          'max_features': 'log2',
          'max_depth': 5}
```

```
In [57]: best_rf_pred = best_rf.predict(X_val)
best_rf_acc = accuracy_score(y_val, rf_pred)

rf_acc
```

Out[57]: 0.8268156424581006

```
In [55]: print("best_rf params:", best_rf.get_params())
print("manual rf params:", rf.get_params())

best_rf params: {'bootstrap': True, 'ccp_alpha': 0.0, 'class_weight': None, 'c
riterion': 'gini', 'max_depth': 5, 'max_features': 'log2', 'max_leaf_nodes': N
one, 'max_samples': None, 'min_impurity_decrease': 0.0, 'min_impurity_split':
None, 'min_samples_leaf': 4, 'min_samples_split': 10, 'min_weight_fraction_lea
f': 0.0, 'n_estimators': 300, 'n_jobs': None, 'oob_score': False, 'random_stat
e': 42, 'verbose': 0, 'warm_start': False}
manual rf params: {'bootstrap': True, 'ccp_alpha': 0.0, 'class_weight': None,
'criterion': 'gini', 'max_depth': None, 'max_features': 'auto', 'max_leaf_node
s': None, 'max_samples': None, 'min_impurity_decrease': 0.0, 'min_impurity_spl
it': None, 'min_samples_leaf': 1, 'min_samples_split': 2, 'min_weight_fraction
_leaf': 0.0, 'n_estimators': 100, 'n_jobs': None, 'oob_score': False, 'random_
state': 42, 'verbose': 0, 'warm_start': False}
```

```
In [58]: (rf_pred == best_rf_pred).mean()
```

Out[58]: 1.0

```
In [59]: test_pred = best_rf.predict(dtest)
test_prob = best_rf.predict_proba(dtest)[:, 1]
submission = pd.DataFrame({
    "PassengerId": dtest_raw["PassengerId"],
    "Survived": test_pred
})
submission.to_csv("submission_rf_improved.csv", index=False)
```

In []: